

1 Editorial: Regate

1.1 Prezentarea problemei

Presupunem că avem N bărci și M poteci (drumuri) între ele, fiecare potecă având un cost. Trebuie să calculăm o valoare optimă pentru fiecare barcă, ținând cont de structura grafurilor și de costurile conexiunilor.

1.2 Observații cheie

1. Poduri în grafuri neorientate

Un pod (bridge) este o muchie a cărei eliminare crește numărul de componente conexe ale grafului. Identificarea podurilor se face cu algoritmul lui Tarjan în $O(N + M)$:

- Parcurgerea DFS cu menținerea nivelului nodului curent
- Pentru fiecare nod v , calculăm $lvlMin[v] = \min\{lvl[w] | w \in \text{subarbore DFS în } v\}$
- Muchia (v, w) este pod dacă și numai dacă $lvlMin[w] > lvl[v]$

2. Descompunerea în componente biconexe

După identificarea podurilor, eliminăm aceste muchii și găsim componentele conexe rămase. Fiecare componentă va fi tratată ca o unitate în pasul următor.

3. Sortarea după costuri

Toate muchiile (podurile) și valorile inițiale sunt sortate în ordine descrescătoare după cost. Aceasta asigură că la fiecare pas, procesăm cea mai avantajoasă conexiune disponibilă.

4. Union-Find cu menținerea valorilor

Structura Union-Find permite combinarea componentelor. Când unim două componente A și B :

$$\begin{aligned} val[newRoot] + &= cost \times cnt[otherRoot] \\ val[otherRoot] + &= cost \times cnt[newRoot] - val[newRoot] \end{aligned}$$

unde val reprezintă diferența cumulativă și cnt dimensiunea componentei.

5. Propagarea valorilor

După construcția arborelui de componente, valorile se propagă de la rădăcină către toate nodurile fiice.

1.3 Algoritmul

1. Se citesc datele de intrare și se construiește graful.
2. Se identifică toate podurile cu algoritmul DFS/Tarjan în $O(N + M)$.
3. Se găsesc componentele conexe rezultate după eliminarea podurilor.
4. Se sortează toate podurile și valorile inițiale după cost în ordine descrescătoare.

5. Se procesează fiecare element sortat:
 - Pentru valori (noduri speciale): se calculează răspunsul parțial
 - Pentru muchii: se unesc componentele folosind Union-Find
6. Se propagă valorile prin arborele rezultat.
7. Se afișează răspunsurile finale pentru fiecare barcă.

1.4 Corectitudine

Lema 1: Algoritmul de găsim podurilor identifică corect toate muchiile a căror eliminare crește numărul de componente conexe.

Lema 2: După eliminarea podurilor, componentele conexe sunt biconexe și maximale.

Lema 3: La fiecare pas de unire, formula de actualizare a valorilor menține invarianții:

$$\sum_{v \in C} val[v] = cost \times dimensiunea\ componentei \times (dimensiunea\ componentei - 1) / 2$$

Lema 4: Propagarea valorilor de la rădăcină către noduri asigură că fiecare nod primește contribuția corectă din toate componentele superioare.

Teorema: Pentru fiecare barcă i , valoarea afișată este corectă conform specificațiilor problemei.

1.5 Complexitate

Complexitatea totală este:

- Găsirea podurilor: $O(N + M)$
- Găsirea componentelor: $O(N + M)$
- Sortarea: $O((N + M) \log(N + M))$
- Operațiile Union-Find cu path compression și union by size: $O((N + M)\alpha(N + M))$

Astfel, complexitatea finală este $O((N + M) \log(N + M))$, suficientă pentru limitele date ($N, M \leq 2 \times 10^5$).

1.6 Note de implementare

```
// Structura pentru muchii
struct edge {
    int x, c, pos; // nod adiacent, cost, pozitie unica
};
```

```
// Structura pentru poduri
struct bridge {
    int x, y, c; // capetele si costul
};
```

- Folosim vectori pentru reprezentarea grafului adiacent
- Matricile *lvl* și *lvlMin* mențin informațiile DFS
- Vectorul *isBridge* marchează muchiile care sunt poduri
- La citire, valorile individuale sunt adăugate în lista de "poduri" cu un marker special ($x = -1$)

1.7 Concluzii

Problema combină elegant conceptele de:

- Identificare a podurilor în grafuri
- Descompunere în componente
- Union-Find cu menținerea valorilor agregate
- Propagare în arbori

Soluția este optimală din punct de vedere al complexității și demonstrează o înțelegere profundă a proprietăților structurale ale grafurilor neorientate.